CORE — Round 2 Assignment

Dental Appointment Booking Agent — Backend Build

Time limit: 48 hours from receipt of this assignment **Eligibility:** Candidates who clear Round 1

About This Round

Round 1 assessed foundational backend skills. This round is designed to reflect the kind of work you'd actually be doing at CORE: architecting and shipping a real, multi-service AI system under a tight timeline, using AI tools as part of your workflow.

We're looking for engineers who can move independently, research unfamiliar APIs on their own, and make sound architectural calls under time pressure — these are the core skills the role requires day to day.

The Brief

Build a working backend for a **Dental Appointment Booking Agent** — the kind of system that would sit behind a voice AI receptionist for a dental clinic.

Core Requirements

1. **Webhook intake** — Accept incoming call webhooks. Simulate the VAPI webhook format. We are not giving you a spec for this — research the actual VAPI webhook structure yourself and build against it.

2. **Multi-turn conversation state** — Manage a conversation flow that progresses through: patient name → service selection → date/time → confirmation. The system needs to track where a given caller is in this flow across multiple webhook hits, not just handle one-shot

requests.

3. **Real calendar booking** — Book the confirmed appointment into Google Calendar using the real Google Calendar API against a test calendar. No mocked calendar logic.

4. **SMS confirmation** — Send a confirmation SMS via Twilio using test credentials once a booking is finalized.

5. **Conversation logging** — Store full conversation logs and state history in Firebase/Firestore.

6. **Admin REST API** — Expose endpoints to view all bookings and the full conversation history per caller/session.

7. **Live deployment** — Deploy the whole thing on Railway, Render, or Vercel. It must be reachable via a public URL, not just running locally.

Ground Rules

- **AI tools are encouraged.** Claude, ChatGPT, Copilot, or any others you use regularly — feel free to use them throughout. We're interested in how effectively you use AI tools to architect and build, not just the end result.
- **Independent research is expected.** Details like VAPI's webhook format, Google Calendar API, Twilio, and Firestore won't be specified for you in advance — researching unfamiliar API surfaces quickly is part of what we're assessing.
- **Language/framework:** Node.js or Python, your choice.
- **Timeline:** 48 hours from when you receive this assignment.

What to Submit

1. **GitHub repo** — public or shared with us, with full commit history visible (we want to see how you built it, not just the final state).
2. **Live URL** — the deployed backend, publicly accessible.

3. **2-minute Loom video** — walk us through your architecture decisions: why you structured the conversation state the way you did, how you'd scale it, and any tradeoffs you made under the time limit.

Evaluation Rubric (100 points)

Area	Pts	What we're looking for
Working end-to-end (live demo)	40	The full flow actually works against the live URL — webhook in, booking lands in calendar, SMS goes out, logs are stored
Architecture (scalable to 1,000 clinics)	20	Would this design hold up multi-tenant? Is state management, data modeling, and service separation sound?
Code quality and organization	15	Readable, sensibly structured, not a single 800-line file
Documentation	15	Another developer should be able to clone this and understand it without asking you questions
Smart use of AI tools to move fast	10	Evidence of using AI tools well to architect and ship quickly — this can come through in the Loom or commit history

Pass Criteria

To pass this round, you need:

- Live deployment that's actually reachable
- All 6 core features functioning (webhook intake, multi-turn state, calendar booking, SMS, Firestore logging, admin API)
- Clean, defensible architecture
- A Loom video that makes it clear you understand what you built, not just that you assembled it

A Note on Approach

There’s no single correct way to structure this system. We’re not grading against a reference implementation — we’re evaluating whether it works end-to-end, whether the architecture would hold up across many clinics, and whether you can clearly explain the decisions you made. If you have to cut a corner under the time limit, mention it in your Loom and tell us what you’d do with more time. That kind of judgment matters as much to us as a complete checklist.

We’re looking forward to seeing what you build.